

Инженерство на подканите и модели за разсъждение

Лекция 10 — Сесия 1

Кратък преговор — Лекция 9

Локални езикови модели:

- **Ollama** и **Llama.cpp** — стек за пускане на LLM локално
- **Квантизация** — намаляване на прецизността на теглата (FP16 → INT8/INT4), за да побере моделът в наличната RAM/VRAM
- Отворени модели — Llama, Qwen, Mistral, DeepSeek
- Кога е смислено да се пуска локално: поверителност, контрол на разходите, експерименти, офлайн употреба

Днес ще използваме Ollama в демото, успоредно с облачен модел.

Какво ще покрием днес

Лекцията е разделена на две сесии:

Сесия 1 (~45 мин) — *тук сме сега*

1. Как работи генерирането на текст
2. Основи на подканите
3. Демо: подкани на живо

Сесия 2 (~50 мин) — след почивката

4. Мисловни вериги (Chain of Thought)
5. Как се обучават моделите за разсъждение
6. Метрики и компромиси по компютърно време
7. Кога какво да използваме — оценяване на подкани и кеширане на подкани (prompt caching)

Сесия 1

Генериране и подкани

Как работи генерирането на текст

Защо този въпрос има значение

Една и съща подкана често дава различни отговори. Защо?

- LLM не извлича отговор от база данни
- Не изчислява детерминистично функция
- Тегли извадка от вероятностно разпределение върху речника

Всичко, което идва после — мисловни вериги, self-consistency, оценяване, моделите за разсъждение — се опира на този факт.

От logits до вероятности

За всеки следващ token моделът произвежда **вектор от logits** — по едно число за всеки token в речника.

Softmax ги превръща във вероятностно разпределение:

$$P(i) = \frac{\exp(z_i)}{\sum_j \exp(z_j)}$$

- z_i — logit за token i
- $P(i)$ — вероятност tokenът да бъде избран
- Сумата на $P(i)$ е равна на 1

[ИЗОБРАЖЕНИЕ: logits-to-probs] — диаграма: последен слой → вектор от logits → softmax → разпределение върху речника.

Greedy decoding

Най-простият начин: винаги избираме токена с най-висока вероятност (argmax).

```
P("котка") = 0.42 ← избираме този  
P("куче")   = 0.31  
P("риба")   = 0.18  
P("...")    = 0.09
```

Свойства:

- Детерминистично — една и съща подкана дава един и същ изход
- На практика: сух текст, повтарящи се фрази, попадане в цикли
- Рядко се използва по подразбиране

Sampling

Вместо `argmax`, **теглим извадка** от разпределението — токени с по-висока вероятност се избират по-често, но не винаги.

```
P("котка") = 0.42 ← избран в 42% от случаите  
P("куче")   = 0.31 ← в 31%  
P("риба")   = 0.18 ← в 18%  
P("...")    = 0.09 ← в 9%
```

Една и съща подкана → различни резултати.

„Креативността“ е статистическо явление, не магия.

Temperature

Скалираме logits преди softmax:

$$P(i) = \frac{\exp(z_i/T)}{\sum_j \exp(z_j/T)}$$

T	Ефект
$T \rightarrow 0$	приближава greedy — детерминистично
$T = 1$	оригиналното разпределение
$T < 1$	изостря — по-консервативни отговори
$T > 1$	изглажда — по-разнообразни, по-рискови отговори

Това не е „креативност на модела“ — само форма на разпределението преди тегленето на извадка.

Top-k и top-p (nucleus sampling)

Преди да теглим извадка, **орязваме** разпределението.

Top-k: запазваме само k -те най-вероятни токени.

Top-p (nucleus): запазваме най-малкото множество токени, чиято кумулативна вероятност е поне p .

Метод	Кога е полезен
top-k = 40	прост, предсказуем
top-p = 0.9	адаптивен — по-малко токени, когато моделът е уверен

Често се комбинират: temperature и top-p заедно контролират шума.

Други параметри

Параметър	Какво прави
<code>max_tokens</code>	спира след N генерирани токена
<code>stop sequences</code>	спира при определен низ (<code>"\n\n"</code> , <code>"User:"</code>)
<code>repetition_penalty</code>	намалява вероятността на вече използвани токени
<code>frequency / presence penalty</code>	подобно, но с различно скалиране

Всеки от тях променя или **разпределението**, или **момента на спиране**.

Какво научихме до тук

- Генерирането е **итеративно теглене на извадки** от разпределение
- Една подкана → много възможни отговори (вариация по дизайн)
- Контролираме процеса с **temperature, top-k, top-p**
- Този факт ще го срещаме многократно в лекцията:
 - Защо мисловните вериги помагат (токени като място за мислене)
 - Защо self-consistency работи (вариация → гласуване)
 - Защо оценяването трябва да отчита шум
 - Защо моделите за разсъждение изразходват много токени

Основи на подканите

Защо подканите изобщо работят

Базовият модел продължава текст. Инструкционният модел (след SFT, Лекция 8) е учен да следва **формати** на асистент:

- въпрос → отговор
- инструкция → изпълнение
- системно съобщение → роля
- примери → продължение по същия шаблон

Инженерството на подканите е изкуството да **попадаме в шаблоните**, на които моделът е обучен.

Когато даден трик „работи магически“, причината обикновено е, че шаблонът е масово срещан в SFT данните.

Подкана без примери (zero-shot)

Описваме задачата на естествен език, без примери.

Преведи на български: "The model produces logits."

Класифицирай настроението (положително / отрицателно / неутрално):
"Филмът беше дълъг и предсказуем."

Работи, защото SFT учи модела да изпълнява инструкции.

Първи избор за прости и често срещани задачи.

Подкана с няколко примера (few-shot)

Показваме няколко примера в подканата, преди реалната заявка.

Английски: hello → Български: здравей
Английски: cat → Български: котка
Английски: book → Български:

Това е **обучение в контекст (in-context learning)** — Лекция 7. Моделът не се преобучава; усвоява шаблона от примерите в текущия контекст.

Полезно за:

- нестандартни формати
- специфични стилове или термини
- задачи, с които подкана без примери не се справя

Подводни камъни при няколко примера

- **Редът на примерите има значение** — последните тежат повече (recency bias)
- **Разпределението на етикетите** — ако всички примери са „положителни“, моделът клони към този етикет
- **Повърхностни шаблони** — моделът може да хване форматирането, не съдържанието
- **Повече примери ≠ по-добре** — след 5–8 примера ефектът намалява, а разходите растат
- **Дължина на контекста** — примерите заемат токени, които иначе биха били за отговора

Ще видим това на живо в демото.

Role / persona

Ти си опитен Python инженер. Отговаряш кратко и с примери от код. Не добавяш предупреждения, освен ако не са необходими.

Защо работи: SFT данните съдържат много диалози с роля. Моделът разпознава формата.

Практически ефекти:

- ТОН И СТИЛ
- дълбочина на отговора (експерт срещу новак)
- подразбиращи се конвенции (примери, форматиране)

Системна подкана

API-тата структурират разговора като списък от съобщения с **роли**:

```
[  
  {"role": "system",    "content": "Ти си български учител..."},  
  {"role": "user",      "content": "Обясни перплексия."},  
  {"role": "assistant", "content": "Перплексия е..."},  
  {"role": "user",      "content": "А с пример?"},  
]
```

- `system` — инструкции с по-висок приоритет; **запазват се през целия разговор**
- `user` / `assistant` — историята на диалога
- Моделът вижда всичко това като един дълъг текст с разделители

Структурирани изходи (structured outputs)

Често искаме **JSON** или конкретна схема, а не свободен текст.

Слаба формулировка:

Извлечи името и възрастта от текста.

По-силна — с конкретен пример:

Извлечи името и възрастта като JSON по схемата:
`{"name": <string>, "age": <integer>}`

Текст: "Иван е на 34 години."
JSON:

Работи, защото SFT данните съдържат структурирани примери. OpenAI и Anthropic имат и API режим, който **гарантира** валиден JSON.

Negative constraints и token economy

Negative constraints — посочете какво **не** искате:

- „Без преамбюл“
- „Не хеджирай — отговори директно“
- „Без излишни обяснения“
- „Едно изречение“

Token economy — всеки токен в подканата струва закъснение и пари:

- Дълга подкана $\neq$ по-добър отговор
- Повтарящите се инструкции рядко помагат
- Добрата подкана е като добър код — **краткостта е признак за яснота**

Демо: подкани на живо

Какво ще покажем

Едни и същи задачи, изпълнени успоредно от **Ollama (локално)** и **облачен модел**.

Прогресия:

1. **Без примери (zero-shot)** — базова линия
2. **+ няколко примера (few-shot)** — наблюдаваме подобрението
3. **+ роля и формат** — по-остър изход
4. **Temperature 0.2 спрямо 1.0** — по два пъти за всяка, виждаме вариацията
5. **JSON изход** — искаме го, чупим го с неясна подкана, оправяме го с примерна схема

Какво да наблюдавате

- **Кои промени правят голяма разлика** — често не са тези, които очакваме
- **Кога примерите вредят** — лошите примери са по-зле от никакви
- **Шумът от temperature** — едни и същи настройки, различни отговори
- **Дължина на отговора** — кога моделът избягва директен отговор и защо

Целта не е „вижте колко добре пишем подкани“, а да видите, че подканите са **код**, който произвежда измерими промени в изхода.

↓ Тетрадка: Демо

Демото продължава в тетрадката

Почивка ~10 мин

След почивката: Сесия 2 — мисловни вериги и модели за разсъждение